

Bloc 2 - Développement d'applications, API REST

TD 3 - Publier une API REST avec Symfony

1. Objectif

L'objectif de cette activité est d'utiliser Symfony et le composant API Platform pour mettre en place une API REST qui publie l'état d'une entité gérée par Doctrine, l'ORM de Symfony.

2. Rappels sur Symfony et Doctrine

Symfony

Symfony est un ensemble de composants PHP, un framework d'application Web, une philosophie et une communauté - tous travaillant ensemble en harmonie.

<https://symfony.com/>

Entité

Les données de la base de données sont des objets. Un objet dont l'enregistrement est confié à l'ORM se nomme une entité (entity en anglais). On dit également persister une entité, plutôt qu'enregistrer une entité.

Doctrine

Symfony est livré par défaut avec l'ORM Doctrine. Un ORM (pour Object-Relation Mapper, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des correspondances entre les schémas de la base de données et les classes du programme applicatif.

Annotations

Les annotations sont des commentaires avec une syntaxe particulière. Doctrine (@ORM) exploite les annotations pour utiliser un objet, créer la table correspondante, l'enregistrer, définir un identifiant (id) en auto-incrément, nommer les colonnes, etc.

Ces informations se nomment les metadata de l'entité. Ajouter les metadata à un objet, s'appelle mapper l'objet, c'est-à-dire faire le lien entre l'objet et la représentation physique qu'utilise Doctrine (une table de la base de données relationnelle)

API Platform

API Platform est un framework basé sur Symfony (et donc MVC) qui permet de développer simplement et rapidement une API REST.

<https://api-platform.com/docs/core/getting-started/>

3. Travail à faire

3.1 Créer le squelette de l'API REST

1. Dans une nouvelle invite de commande sous votre racine web www, créez un nouveau projet Symfony sans utiliser l'option `--full` :

```
C:\wamp64\www>symfony new ludo-api
```

2. Déplacez-vous dans le dossier de l'application et démarrez le serveur Symfony

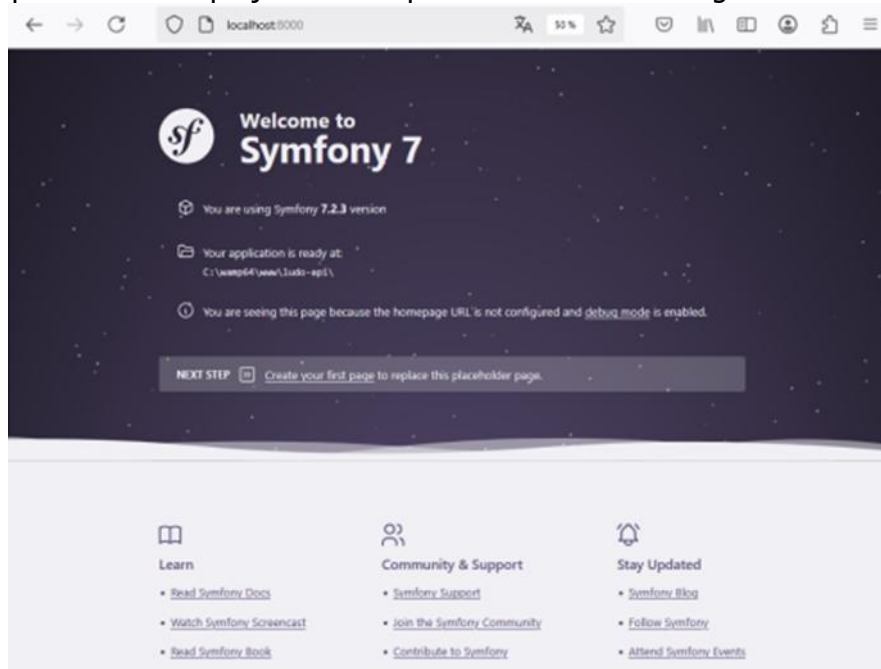
```
C:\wamp64\www>cd ludo-api
```

```
C:\wamp64\www\ludo-api> symfony serve
```

Rappels :

- symfony serve est l'équivalent de symfony server:start
- l'option -d permet de lancer la commande en mode detach pour pouvoir réutiliser le même terminal
- l'option --no-tls sert à désactiver l'encryption TLS qui est utilisée par défaut.

3. Constater que le nouveau projet est bien publié à l'aide d'un navigateur web :



4. Dans une nouvelle invite de commande sous votre application, mettez à jour composer avec :

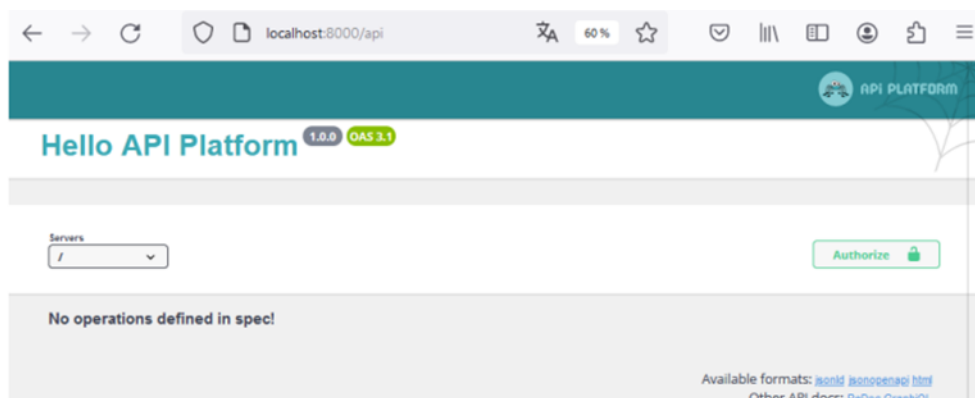
```
C:\wamp64\www\ludo-api> composer selfupdate
```

5. Installez Api Platform avec composer, répondez non (n) pour la configuration Docker

```
C:\wamp64\www\ludo-api>composer require api
```

```
...
This may create/update compose.yaml or update Dockerfile (if it exists).
Do you want to include Docker configuration from recipes?
[y] Yes
[n] No
[p] Yes permanently, never ask again for this project
[x] No permanently, never ask again for this project
(defaults to y): n
...
```

6. Dans le navigateur, vérifier que Api Platform est désormais actif à l'URL localhost:8000/api :



API Platform utilise le format OpenAPI pour décrire les endpoints de notre API et l'interface graphique Swagger UI permettant de visualiser et d'interagir avec l'API.

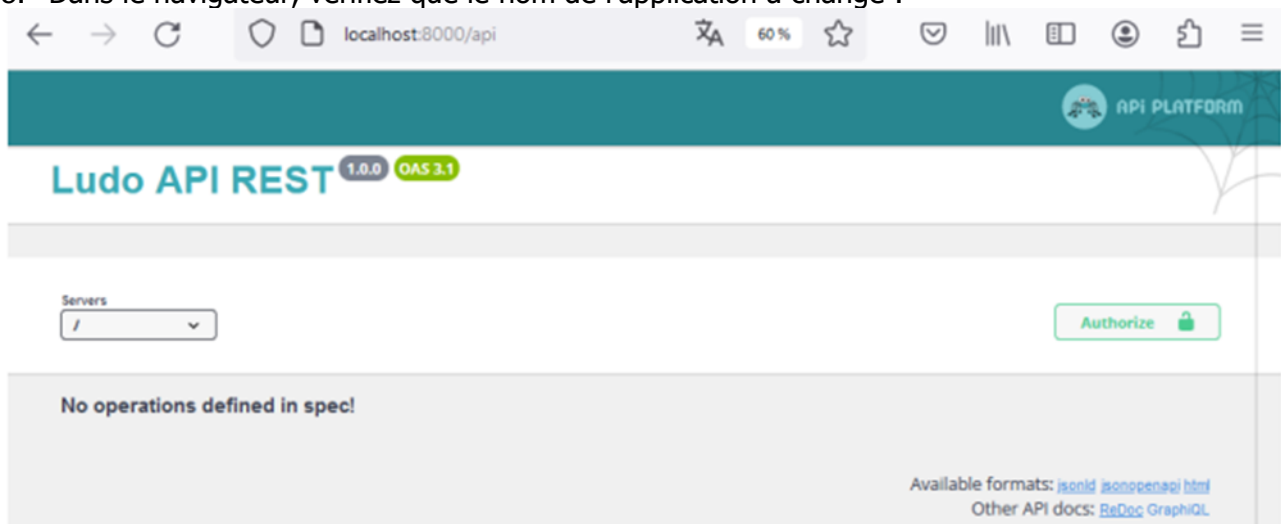
Un point de terminaison d'API est un emplacement numérique exposé via l'API à partir duquel l'API reçoit des requêtes et envoie des réponses. Pour l'instant nous n'en avons pas encore créé.

7. Ajoutez le nom de l'application à cette page. Pour cela, modifiez le titre dans le fichier .yml du dossier config/packages comme suit :

Fichier ludo-api/config/packages/api_platform.yml

```
api_platform:
  title: Ludo API REST
  version: 1.0.0
  defaults:
    stateless: true
    cache_headers:
      vary: ['Content-Type', 'Authorization', 'Origin']
```

8. Dans le navigateur, vérifiez que le nom de l'application a changé :



3.2 Création de la ressource à publier

1. Une API REST manipule des ressources, il s'agit ici des entités. Nous utiliserons le Makerbundle pour créer les entités. Commençons par installer MakerBundle :

```
C:\wamp64\www\ludo-api>composer require symfony/maker-bundle --dev
```

2. Ensuite, créons l'entité Mot en précisant que nous souhaitons en faire une ressource de l'API :

```
C:\wamp64\www\ludo-api>php bin/console make:entity
```

```
Class name of the entity to create or update (e.g. AgreeableChef):
```

```
> Mot
```

```
Mark this class as an API Platform resource (expose a CRUD API for it) (yes/no) [no]:
```

```
> yes
```

```
created: src/Entity/Mot.php
```

```
created: src/Repository/MotRepository.php
```

```
Entity generated! Now let's add some fields!
```

```
You can always add more fields later manually or by re-running this command.
```

```
New property name (press <return> to stop adding fields):
```

```
> contenu
```

```
Field type (enter ? to see all types) [string]:
```

```
>
Field length [255]:
> 40
Can this field be null in the database (nullable) (yes/no) [no]:
>
  updated: src/Entity/Mot.php
Add another property? Enter the property name (or press <return> to stop adding fields):
> nbPoints
Field type (enter ? to see all types) [string]:
> integer
Can this field be null in the database (nullable) (yes/no) [no]:
>
  updated: src/Entity/Mot.php
Add another property? Enter the property name (or press <return> to stop adding fields):
> dureeSec
Field type (enter ? to see all types) [string]:
> integer
Can this field be null in the database (nullable) (yes/no) [no]:
> yes
  updated: src/Entity/Mot.php
Add another property? Enter the property name (or press <return> to stop adding fields):
> dateCreation
Field type (enter ? to see all types) [string]:
> datetime
Can this field be null in the database (nullable) (yes/no) [no]:
>
  updated: src/Entity/Mot.php
Add another property? Enter the property name (or press <return> to stop adding fields):
> dateModification
Field type (enter ? to see all types) [string]:
> datetime
Can this field be null in the database (nullable) (yes/no) [no]:
> yes
  updated: src/Entity/Mot.php
Add another property? Enter the property name (or press <return> to stop adding fields):
>
  Success!
Next: When you're ready, create a migration with php bin/console make:migration
C:\wamp64\www\ludo-api>
```

3. Le fichier `src/Entity/Mot.php` a été créé ainsi que `src/Repository/MotRepository.php`. Grâce aux annotations, chaque propriété est mappée (mise en correspondance) avec une colonne de la table de la base de données. Les accesseurs (getters et setters) ont également été générés. Remarquez l'annotation qui indique que l'entité `Mot` est une ressource de l'API :

Fichier ludo-api/src/Entity/Mot.php

<?php

namespace App\Entity;

use ApiPlatform\Metadata\ApiResource;

use App\Repository\MotRepository;

use Doctrine\DBAL\Types\Types;

use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: MotRepository::class)]

#[ApiResource]

class Mot

{

#[ORM\Id]

#[ORM\GeneratedValue]

#[ORM\Column]

private ?int \$id = null;

#[ORM\Column(length: 40)]

private ?string \$contenu = null;

#[ORM\Column]

private ?int \$nbPoints = null;

#[ORM\Column(nullable: true)]

private ?int \$dureeSec = null;

#[ORM\Column(type: Types::DATETIME_MUTABLE)]

private ?\DateTimeInterface \$dateCreation = null;

#[ORM\Column(type: Types::DATETIME_MUTABLE, nullable: true)]

private ?\DateTimeInterface \$dateModification = null;

public function getId(): ?int

{

return \$this->id;

}

public function getContenu(): ?string

{

return \$this->contenu;

}

public function setContenu(string \$contenu): static

{

\$this->contenu = \$contenu;

return \$this;

}

public function getNbPoints(): ?int

{

return \$this->nbPoints;

}

utilisation de APIPlatform

l'annotation qui indique que l'entité
Mot est une ressource de l'API

```
public function setNbPoints(int $nbPoints): static
{
    $this->nbPoints = $nbPoints;

    return $this;
}

public function getDureeSec(): ?int
{
    return $this->dureeSec;
}

public function setDureeSec(?int $dureeSec): static
{
    $this->dureeSec = $dureeSec;

    return $this;
}

public function getDateCreation(): ?\DateTimeInterface
{
    return $this->dateCreation;
}

public function setDateCreation(\DateTimeInterface $dateCreation): static
{
    $this->dateCreation = $dateCreation;

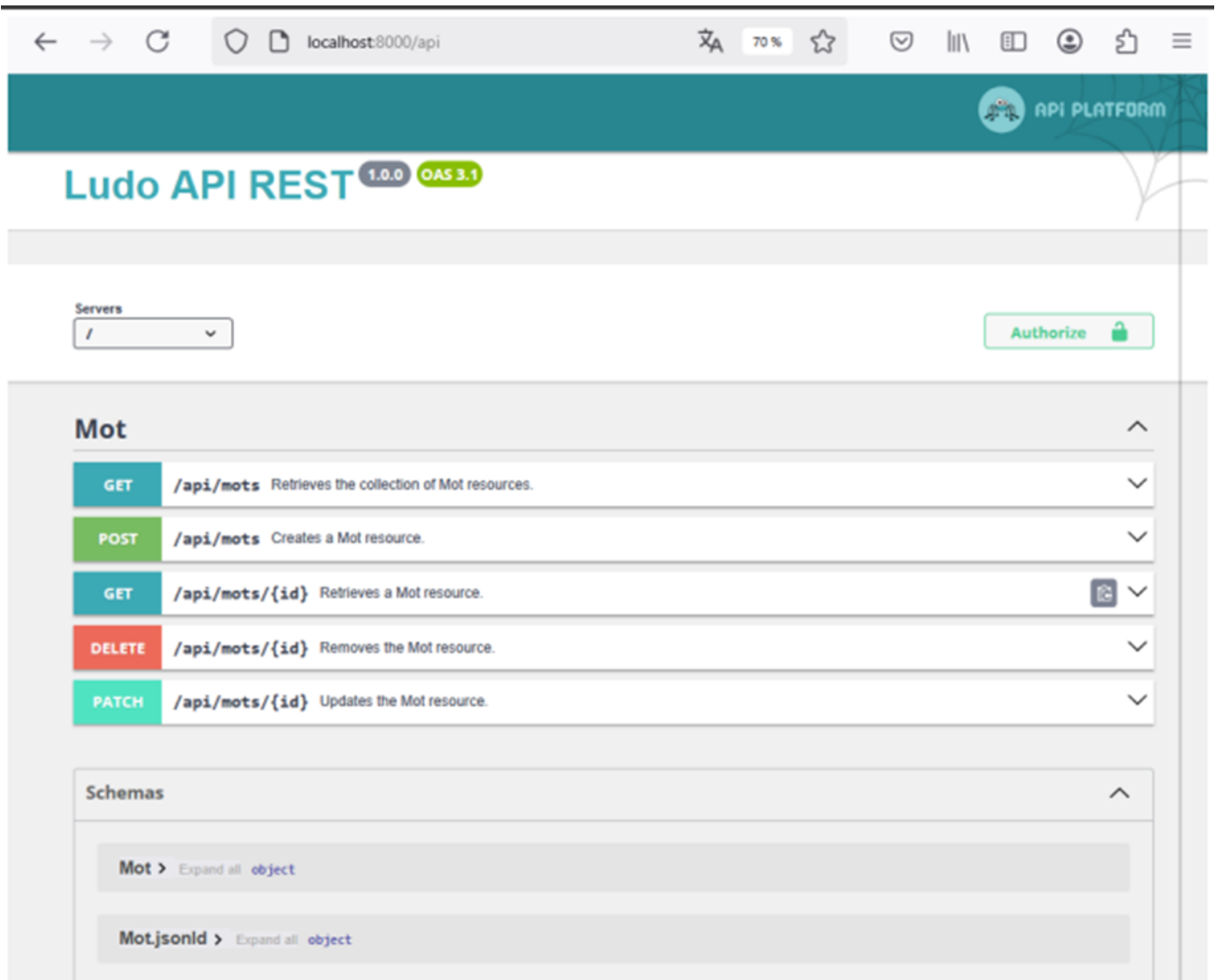
    return $this;
}

public function getDateModification(): ?\DateTimeInterface
{
    return $this->dateModification;
}

public function setDateModification(? \DateTimeInterface $dateModification): static
{
    $this->dateModification = $dateModification;

    return $this;
}
}
```

4. Dans le navigateur, vérifier à l'URL localhost:8000/api que Api Platform publie désormais la ressource mot et qu'il a généré automatiquement toutes les opérations (GET...) pour cette nouvelle ressource.



Un message d'erreur apparaît si l'on tente de solliciter le endpoint /api/mots car la migration de l'entité en BDD n'a pas encore été menée, le endpoint GET ne peut donc pas rendre l'état de la ressource :



5. Pour créer la nouvelle entité mot en base de données, il faut renseigner la base de données dans le fichier .env de l'application.
pour cela, adaptez le fichier .env à votre environnement (root, mdp, version) :

Fichier ludo-api/.env

```
# In all environments, the following files are loaded if they exist,
# the latter taking precedence over the former:
#
# * .env                contains default values for the environment variables needed by the app
# * .env.local          uncommitted file with local overrides
# * .env.$APP_ENV       committed environment-specific defaults
# * .env.$APP_ENV.local uncommitted environment-specific overrides
#
# Real environment variables win over .env files.
#
# DO NOT DEFINE PRODUCTION SECRETS IN THIS FILE NOR IN ANY OTHER COMMITTED FILES.
# https://symfony.com/doc/current/configuration/secrets.html
#
# Run "composer dump-env prod" to compile .env files for production use (requires symfony/flex
# >=1.2).
# https://symfony.com/doc/current/best_practices.html#use-environment-variables-for-
# infrastructure-configuration

###> symfony/framework-bundle ###
```

```
APP_ENV=dev
APP_SECRET=
###< symfony/framework-bundle ###

###> doctrine/doctrine-bundle ###
# Format described at https://www.doctrine-project.org/projects/doctrine-
# dbal/en/latest/reference/configuration.html#connecting-using-a-url
# IMPORTANT: You MUST configure your server version, either here or in
# config/packages/doctrine.yaml
#
# DATABASE_URL="sqlite:///kernel.project_dir%/var/data.db"
# DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=8.0.32&charset=utf8mb4"
# DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=10.11.2-
# MariaDB&charset=utf8mb4"
# DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=16&charset=utf8"
DATABASE_URL="mysql://root:@127.0.0.1:3306/ludo?serverVersion=8.0.31"
###< doctrine/doctrine-bundle ###

###> nelmio/cors-bundle ###
CORS_ALLOW_ORIGIN='^https://(localhost|127\.\0\.\0\.\1)(:[0-9]+)?$'
###< nelmio/cors-bundle ###
```

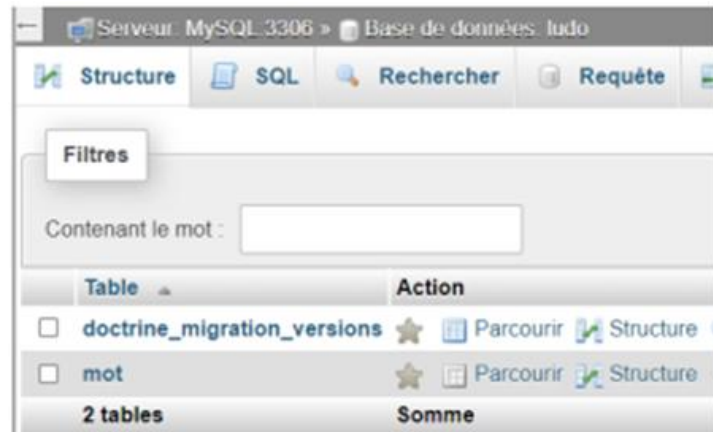
6. Démarrer le serveur MySQL
7. Créer la base de données à l'aide de doctrine :

```
C:\wamp64\www\ludo-api> php bin/console doctrine:database:create
```

8. Générer et exécuter les migrations :

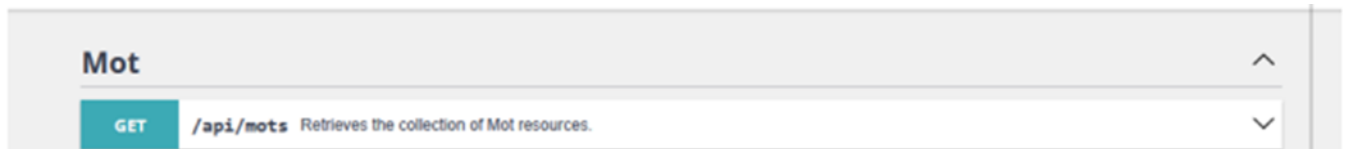
```
C:\wamp64\www\ludo-api> php bin/console make:migration
C:\wamp64\www\ludo-api> php bin/console doctrine:migrations:migrate
```


9. Vérifiez que la table mot a bien été créée avec phpAdmin :

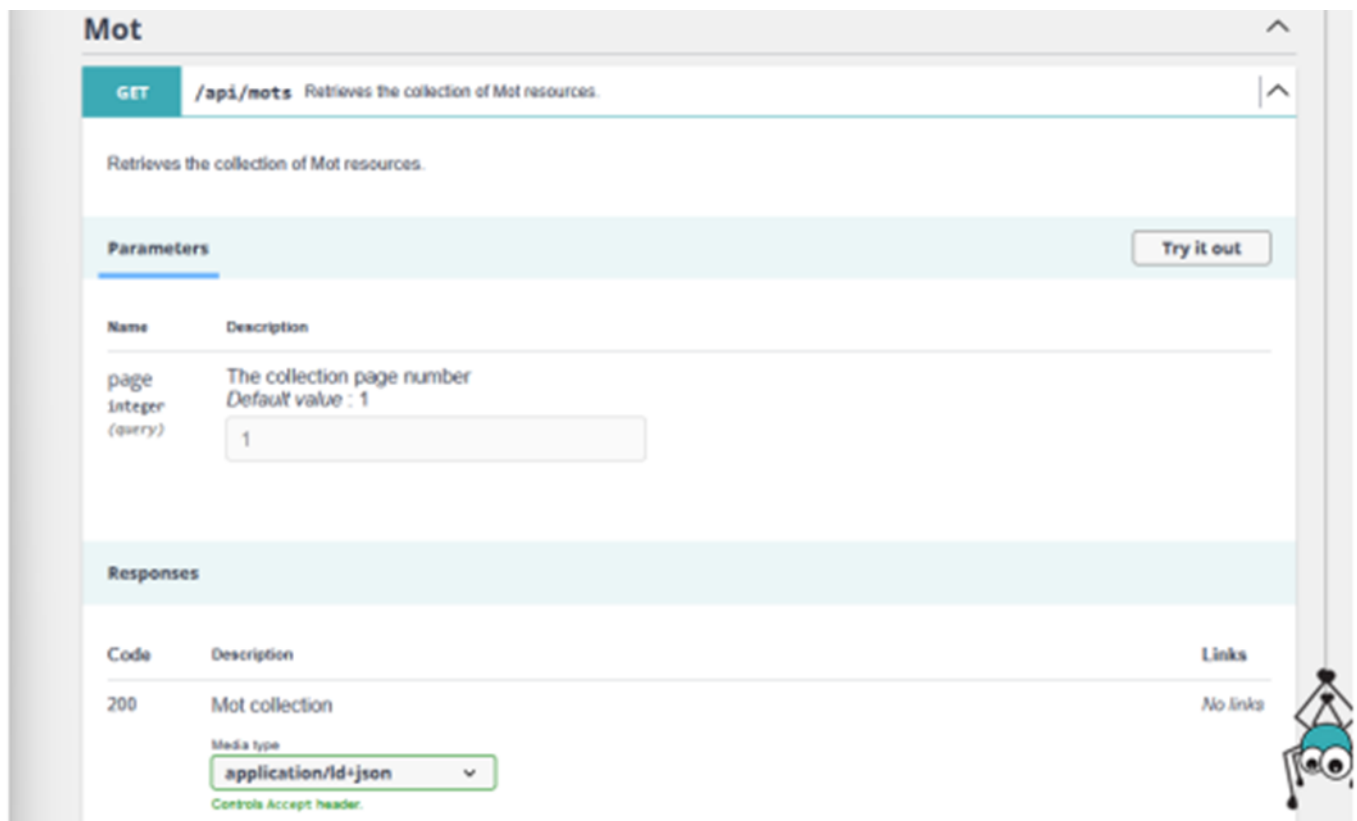


3.3 Test des Endpoints

1. Testez les endpoints de l'API avec Swagger UI/APIPlatform.
Sur la page localhost:8000/api, cliquez sur le premier Endpoint (GET) /api/mots :



Cela révèle une section de détail pour le Endpoint GET /api/mots :



Ce premier Endpoint GET permet de récupérer tous les mots.

Dans la section **Parameters**, il est indiqué que la requête prend un paramètre **page** de type entier (integer) et que ce paramètre va être dans la requête (query).

En face, la description indique que ce paramètre page représente le numéro de page de la collection et la valeur par défaut est 1. Il s'agit d'un système de pagination pour éviter de renvoyer un nombre trop volumineux de données. Par exemple s'il y a 100 000 mots dans la base.

Par défaut API Platform renvoie 30 ressources par page, et il est possible de modifier cette valeur ou de désactiver la pagination. Quand l'utilisateur aura besoin de la 2e page, il enverra la requête GET /api/mots?page=2 et ainsi de suite.

2. Pour tester le Endpoint, cliquez sur le bouton "Try it out". Laissez la valeur par défaut du paramètre page à 1 et cliquez ensuite sur le bouton "Execute".

Une commande curl est exécutée, vous obtenez un message 200 mais un JSON précisant qu'il n'y a pas d'enregistrements disponibles :

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8000/api/mots?page=1' \
-H 'accept: application/ld+json'
```

Request URL

```
http://localhost:8000/api/mots?page=1
```

Server response

Code	Details
200	

Response body

```
{
  "@context": "/api/contexts/Mot",
  "@id": "/api/mots",
  "@type": "Collection",
  "totalItems": 0,
  "member": []
}
```

la réponse [] est vide

Response headers

```
cache-control: no-cache, private
content-length: 103
content-type: application/ld+json; charset=utf-8
date: Thu, 13 Feb 2025 19:37:57 GMT
etag: "e9b20b95a98bd11c"
link: <http://localhost:8000/api/docs.jsonld>; rel="http://www.w3.org/ns/hydra/core#apiDocumentation"
vary: Accept, Content-Type, Authorization, Origin
x-content-type-options: nosniff
x-frame-options: deny
x-powered-by: PHP/8.2.0
x-robots-tag: noindex
```

La réponse est vide, ce qui est normal puisqu'il n'y a pas encore de mots dans la base de données.

3. Pour ajouter un mot, utilisez le Endpoint **POST /api/mots**

Cliquez sur POST puis sur le bouton « **Try it out** ». Saisir le JSON permettant de décrire le nouveau mot dans le champ « Request body ». Les dates peuvent être laissées aux valeurs générées :

POST /api/mots Creates a Mot resource.

Creates a Mot resource.

Parameters

No parameters

Request body *required* application/json

The new Mot resource

```
{
  "contenu": "hortensia",
  "nbPoints": 3,
  "dureeSec": 60,
  "dateCreation": "2025-02-13T19:48:46.726Z",
  "dateModification": "2025-02-13T19:48:46.726Z"
}
```

Execute

Cliquez sur le bouton « Execute ». L'API retourne un code 201 (created) en réponse (Mot ressource created).

Responses

Code	Description
201	Mot resource created

Media type application/json

Controls Accept header.

Example Value Schema

```
{
  "@context": "string",
  "@id": "string",
  "@type": "string",
  "id": 0,
  "contenu": "string",
  "nbPoints": 0,
  "dureeSec": 0,
  "dateCreation": "2025-02-13T19:41:49.875Z",
  "dateModification": "2025-02-13T19:41:49.875Z"
}
```

Vérifier que le mot a bien été créé en base de données.

Serveur : MySQL:3306 » Base de données : ludo » Table : mot

Parcourir Structure SQL Rechercher Insérer Exporter Plus

✓ Affichage des lignes 0 - 0 (total de 1, traitement en 0,0022 seconde(s).)

SELECT * FROM `mot`

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table

Options supplémentaires

id	contenu	nb_points	duree_sec	date_creation	date_modifi
1	hortensia	3	60	2026-02-25 11:37:36	NULL

4. Cliquer et exécuter de nouveau le Endpoint (GET) /api/mots , l'API rend désormais la liste des enregistrements de la ressource mot, soit celui qui vient d'être créé :

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/mots?page=1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/mots?page=1
```

Server response

Code	Details
200	Response body <pre>{ "context": "/api/contexts/Mot", "id": "/api/mots", "type": "Collection", "totalItems": 1, "member": [{ "id": "/api/mots/1", "type": "Mot", "id": 1, "content": "hortensia", "nbPoints": 2, "dureeSec": 60, "dateCreation": "2025-02-13T19:40:46+00:00", "dateModification": "2025-02-13T19:40:46+00:00" }] }</pre>

Download

Pour modifier un des enregistrements de la ressource, cliquer sur PATCH et sur le bouton « Try it out » :

PATCH /api/mots/{id} Updates the Mot resource.

Updates the Mot resource.

Parameters

Try it out

Saisir l'ID de l'enregistrement à modifier et renseigner le JSON en précisant les nouvelles valeurs à modifier dans le champ « Request body ». Dans l'exemple ci-dessous, seulement nbPoints va être modifié:

PATCH /api/mots/{id} Updates the Mot resource.

Updates the Mot resource.

Parameters

Cancel Reset

Name	Description
id * required	Mot identifier
string (path)	1

Request body required

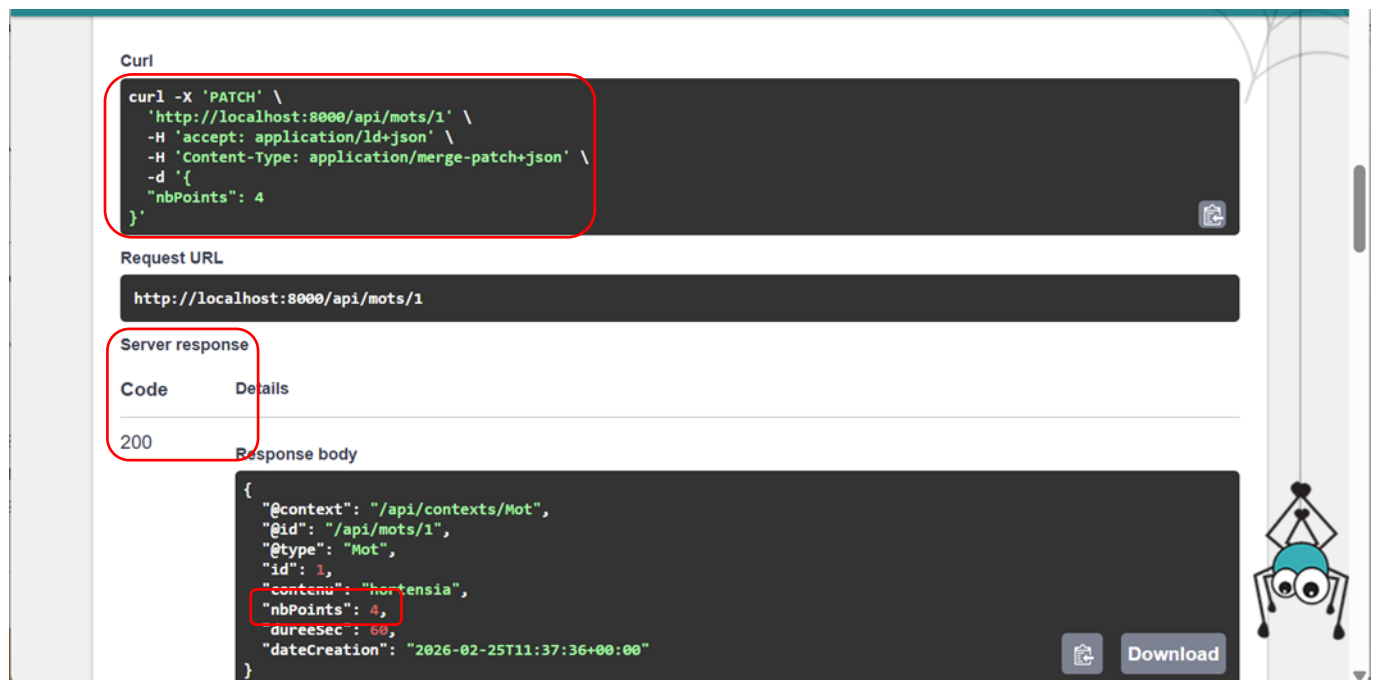
application/merge-patch+json

The updated Mot resource

```
{
  "nbPoints": 4
}
```

et cliquer sur le bouton « Execute ».

Relever la commande CURL passé par API Platform, ainsi que la réponse « 200 » qui confirme que la modification a bien été prise en compte et les nouvelles valeurs de l'enregistrement :



Confirmer la prise en compte de la modification directement BDD :



Enfin, tester le point de terminaison DELETE et cliquer sur le bouton « Try it out » :

The screenshot shows the API Platform interface with a list of endpoints. The **DELETE** endpoint for `/api/mots/{id}` is highlighted with a red box. Below the endpoint list, the **Parameters** section is visible, and a **Try it out** button is highlighted with a red box.

Spécifier l'Id de l'enregistrement à supprimer puis cliquer sur le bouton « Execute » :

The screenshot shows the **DELETE** endpoint for `/api/mots/{id}` with the **Parameters** section expanded. The **id** parameter is highlighted with a red box, showing it is a required string (path) with the value `1` entered. The **Execute** button is also highlighted with a red box.

Relever la commande CURL passée par APIPlatform et le message 204 retournée par l'API :

The screenshot shows the **Responses** section of the API Platform interface. The **Execute** button is highlighted with a red box. Below it, the **Curl** command is displayed and highlighted with a red box: `curl -X 'DELETE' \ 'http://localhost:8000/api/mots/1' \ -H 'accept: */*'` . The **Request URL** is `http://localhost:8000/api/mots/1`. The **Server response** section is expanded, showing a **Code** of `204` (highlighted with a red box) and the **Response headers** listed below.

Vérifier la suppression en BDD :

The screenshot shows the phpMyAdmin web interface. On the left sidebar, the 'ludo' database is selected, and the 'mot' table is highlighted. The main panel displays the 'Structure' tab for the 'mot' table. A green message box at the top indicates: 'MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0004 seconde(s).)'. Below this, the SQL query 'SELECT * FROM `mot`' is entered. The table structure is shown with columns: id, contenu, nb_points, duree_sec, date_creation, and date_modification. The bottom section is labeled 'Opérations sur les résultats de la requête'.

phpMyAdmin

Serveur courant : MySQL

Récentes Préférées

ludo

Nouvelle table

doctrine_migration_version

mot

Serveur : MySQL:3306 » Base de données : ludo » Table : mot

Parcourir Structure SQL Rechercher Insérer Exporter Importer

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0,0004 seconde(s).)

SELECT * FROM `mot`

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

id contenu nb_points duree_sec date_creation date_modification

Opérations sur les résultats de la requête

4. Test des endpoints de l'API avec Postman

Il est possible d'utiliser d'autres logiciels que APIPlatform pour tester les API à la condition qu'elles soient REST.

Postman, est l'un de ces outils de test des API REST.

4.1 Installation et utilisation de Postman

1. Télécharger l'application Postman à partir de : <https://www.postman.com/downloads/> et installer-la.
2. Pour une utilisation simple, il n'est pas nécessaire de créer un compte ou d'utiliser son compte Google, il

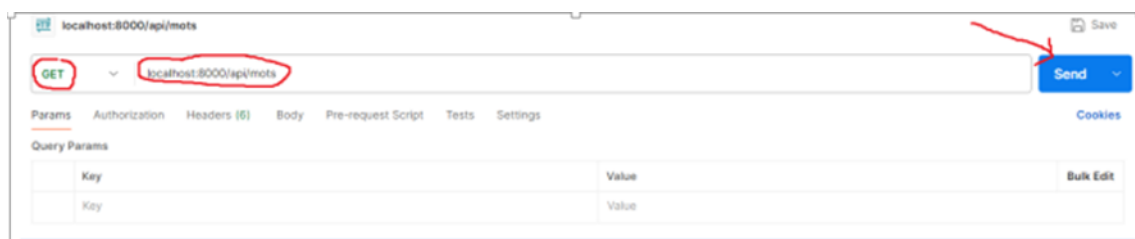
est possible de cliquer sur :

Open Lightweight API Client

3. Renseigner le formulaire en partie haute du logiciel Postman :

- choisir la méthode GET
- renseigner l'URI : localhost:8000/api/mots

et appuyer sur le bouton « Send » :



4. Observez la réponse de l'API dans la partie basse du logiciel :



4.2 Les opérations

Une opération représente le lien entre une ressource, une route et le contrôleur associé.

Par défaut, API Platform définit les opérations CRUD sur toutes les ressources exposées par l'API. C'est le cas pour la ressource « Mot ». Il est possible de créer un mot, le modifier, le supprimer et lire la liste des mots.

On distingue deux types d'opérations :

- les opérations sur les collections (collectionOperations)
- les opérations sur une entité/une ressource (itemOperations).

Les opérations sur les collections s'appliquent aux collections de ressources. Deux méthodes HTTP sont implémentées par défaut :

- GET /api/mots pour récupérer la liste des ressources
- POST /api/mots pour ajouter une nouvelle ressource

Les opérations sur une ressource s'appliquent sur une ressource et 4 méthodes HTTP sont implémentées :

- GET — GET /api/mots/{id} pour récupérer une seule ressource dont l'id est mentionné
- PUT — PUT /api/mots/{id} pour remplacer la ressource dont l'id est mentionné
- PATCH — PATCH /api/mots/{id} pour modifier la ressource dont l'id est mentionné
- DELETE — DELETE /api/mots/{id} pour supprimer la ressource dont l'id est mentionné

Il est possible d'activer ou de désactiver certaines opérations. Testez les différentes opérations !

5. Conclusion de l'activité

Cette activité a permis de mettre en place un API qui publie l'état de la ressource jeu en utilisant le framework APIPlatform qui se base sur Symfony.